

移动应用供应链隐私合规 与 LLM 智能体程序分析

从 DIULENS 到 PICOSCAN、GPTDroid 与 IRIS

从 DIULENS 出发的研究地图

同一条隐私承诺，要在界面、配置、代码和运行时同时成立

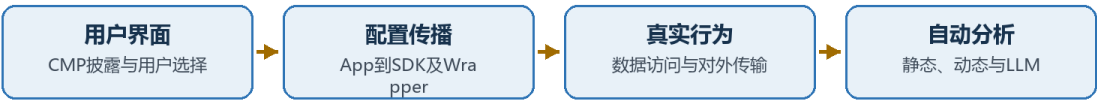


图1 学习主线：界面承诺、SDK 配置、运行行为与自动分析

使用建议 第一遍只读每章的“先记住一句话”和图；第二遍精读三篇核心论文；第三遍使用讲解卡片练习向老师复述。

编制日期：2026 年 8 月 | 适用对象：软件工程背景、网络安全零基础的研一学生

说明：本文中的“精译”是基于论文原意的中文重述，用于学习，不是逐句官方译文。

阅读导航

- 第一部分：先建立共同语言——SDK、CMP、Consent Signal、静态分析、动态分析与污点分析。
- 第二部分：以 DIULENS 为坐标原点，明确当前论文处在整条供应链的哪个位置。
- 第三至第五部分：精读 PICOSCAN、GPTDroid、IRIS，分别掌握隐私配置分析、LLM GUI 测试和 LLM 辅助静态分析。
- 第六部分：六篇扩展论文的阅读价值与应读章节。
- 第七部分：横向对比、思考题、八周学习路线和可复现 Demo 方向。
- 附录：给老师讲论文的通用模板、阅读笔记模板、术语表和正式来源。

最短路线 时间有限时，只读：术语表 → DIULENS 定位 → PICOSCAN 原则与 PVP → GPTDroid 双循环 → IRIS 四阶段 → 总对比表。

版权与使用边界

本手册没有整篇复制或逐句翻译论文。精读章节提供摘要原意精译、关键定义翻译、方法与实验的逐节中文讲解，并明确哪些内容属于作者结论、哪些属于本手册的解释或推断。正式引用时，应回到论文原文核对术语、表格、样本量和数值。

本手册收录的论文

层级	系统/简称	出处	核心问题
坐标	DIULENS	IEEE S&P 2026	CMP GUI 与有效同意
精读 1	PICOSCAN	USENIX Security 2024	SDK 隐私配置与 Wrapper 传播
精读 2	GPTDroid	ICSE 2024	LLM 驱动移动 GUI 动态测试
精读 3	IRIS	ICLR 2025	LLM 辅助全仓库静态污点分析

层级	系统/简称	出处	核心问题
导读	PESP	USENIX Security 2024	SDK 之间的数据隔离
导读	Covert Identifiers	USENIX Security 2024	追踪 SDK 与动态监控
导读	InconPreter	NDSS 2025	用户预期与实际行为不一致
导读	LLift	OOPSLA 2024	LLM 降低静态分析误报
导读	CoverUp	FSE 2025	覆盖率反馈驱动测试生成
导读	MAdroid	ICSE 2026	多智能体、多设备协同测试

第一部分 零基础先修：先把概念连成一条链

先记住一句话 隐私合规不是“App 有没有弹窗”，而是用户看到的承诺、程序传递的配置、SDK 执行的行为和网络/文件证据必须相互一致。

1.1 一个具体场景

假设一款记账 App 集成了统计 SDK、广告 SDK 和社交登录 SDK。首次启动时，CMP 显示“是否允许个性化广告”。用户选择拒绝后，系统理想状态应当完成四件事：

- CMP 把“拒绝个性化广告”编码成明确的 Consent Signal。
- App 在广告 SDK 初始化之前读取该信号，并调用正确的隐私配置 API。
- 如果广告 SDK 内部还封装了其他 SDK，Wrapper 应继续向下传播相同配置。
- 运行时广告 SDK 不应继续访问或传输用于个性化追踪的数据。

四个环节中任何一个失效，都可能出现“界面上拒绝了，代码或 SDK 却没有真正拒绝”的合规风险。

1.2 必备术语

术语	零基础解释
第三方 SDK	开发者引入的现成代码组件，例如广告、统计、支付、地图、社交登录。它通常运行在 App 进程里，却由外部公司维护。
供应链	App 并不是只有开发者自己的代码，还依赖 SDK、SDK 的子依赖、云端配置和后端服务。责任与行为沿这条链传播。
CMP	Consent-Management Platform，同意管理平台。它通过 GUI 收集、保存并分发用户的同意状态。
Consent Signal	机器可读取的同意状态，例如布尔值、枚举值或 TCF Consent String。它不是弹窗本身，而是弹窗操作产生的程序状态。
Wrapper SDK	一个对外提供统一接口、内部再封装多个 SDK 的组件。它必须把隐私选择正确传给所有内部 SDK。
静态分析	不运行程序，分析 APK、字节码、调用关系、控制流和数据流。覆盖面大，但容易受反射、动态加载、混淆影响。
动态分析	真正运行 App 并观察 UI、API、文件、网络和时间事件。证据更直接，但一次执行只能覆盖被走到的路径。
Source	污点数据的来源，例如 HTTP 请求参数、设备标识符、定位信息。
Sink	敏感数据到达后可能产生危险的位置，例如网络发送、命令执行、文件写入。
Sanitizer	在 Source 到 Sink 之间净化或验证数据的操作，例如路径规范化、转义、白名单检查。
LLM 智能体	LLM 不仅生成文字，还能读取环境状态、选择工具、执行动作、接收反馈并继续决策。
Oracle	测试判定器：负责判断一次执行是否存在缺陷。崩溃、规则违规、覆盖率变化都可成为 Oracle。

1.3 静态、动态和 LLM 分别擅长什么

方法	擅长	主要短板	在本文中的角色
静态分析	大规模扫描全部可见代码路径	误报；难处理动态加载/反射	PICOSCAN、IRIS、LLift
动态分析	提供真实运行证据与时序	覆盖不完整；环境成本高	DIULENS、隐蔽标识符
LLM	理解 UI、API 文档、代码语义	会幻觉；结果不稳定；成本	GPTDroid、IRIS、MAdroid
混合分析	将覆盖能力和语义能力结合	系统复杂、验证成本高	本方向的主流路线

判断标准 只要论文让 LLM 作出判断，就应追问：输入证据是什么？输出如何约束？有没有传统工具复核？结果能否重复？

第二部分 坐标原点：DIULENS 在研究链路中的位置

论文：Understanding and Analyzing Privacy Risks in Mobile Consent-Management Platforms, IEEE Symposium on Security and Privacy 2026。作者为 Jingzhou Ye、Fares Alharbi、Luyi Xing、Xueqiang Wang。

一句话 DIULENS 检查的是：移动 App 通过 CMP 获得的用户同意，是否在展示时机、披露内容、可撤回/不强迫以及交互效果上真正有效。

2.1 DIULENS 的完整工作链



这里的 LLM 不是替代所有分析。SDK 识别、调用轨迹和 UI 自动执行由确定性工具完成；LLM 主要用于理解界面语义、选择探索方向以及把多种证据映射到规则。

2.2 四条规则如何覆盖有效同意

规则	回答的问题	核心证据
Rule 1 时机	收集数据前是否先获得同意？	CMP 首次出现时间、SDK 数据访问时间
Rule 2 可信披露	界面宣称的 SDK 是否与实际集成一致？	截图/文本、真实 SDK 清单
Rule 3 可撤回且不强迫	用户能否改变选择，能否拒绝非必要处理？	入口路径、控件状态、拒绝路径
Rule 4 效果明确	点击和导航是否产生一致、可理解的同意结果？	交互序列、前后状态、Consent Signal

研究边界也很重要：这四条是针对终端用户与 CMP GUI 交互时可观察的最低要求，并不声称覆盖后端如何保存、同步或二次使用 Consent Signal。

2.3 DIULENS 与后续论文的关系

DIULENS 已经回答	仍然没完全回答	对应后续论文
CMP 界面是否正确、可达和可理解	隐私 API 是否被正确调用	PICOSCAN

DIULENS 已经回答	仍然没完全回答	对应后续论文
LLM 如何找到并探索 CMP	通用移动 GUI 测试如何维持长期目标	GPTDroid
实际 SDK 与界面披露是否一致	SDK 内部是否窃取/共享其他组件数据	PESP
规则证据如何综合	代码级漏洞 Source-Sink 如何自动发现	IRIS

DIULENS 论文: <https://xw48.github.io/files/ye2026understanding.pdf>

第三部分 精读一：PICOSCAN——移动 SDK 隐私配置供应链

论文：Navigating the Privacy Compliance Maze: Understanding Risks with Privacy-Configurable Mobile SDKs。USENIX Security 2024。

先记住一句话 给 SDK 增加隐私配置 API 并不等于完成合规；App 可能漏调、错调，SDK 可能默认不隐私，Wrapper 还可能传播失败。

PICOSCAN：把原则落成可扫描的代码模式

静态分析为主，个别默认值和运行行为由动态分析补充



图2 PICOSCAN 的知识库、两类检查器和违规模式

3.1 摘要原意精译

随着 GDPR 和 CCPA 等法规出现，移动 App 必须处理隐私合规问题，但第三方 SDK 的数据行为往往不透明。许多 SDK 开始提供隐私 API，让 App 配置其数据收集、共享或出售行为。论文把这类 SDK 称为 Privacy-Configurable SDK，也就是 PICO SDK。作者提出 PICOSCAN，自动分析 App 和 SDK 代码，并研究隐私配置是否被正确使用、SDK 承诺是否被真正执行、Wrapper 是否正确向内部 SDK 传播配置。对 65 个常用 PICO SDK 和 48,305 个 Google Play App 的测量表明，隐私配置在设计、实现和使用各环节都可能失败。

这段摘要的重点不是“发现很多有问题的 App”，而是提出了一个供应链责任模型：宿主 App、PICO SDK、PICO Wrapper 三类角色分别承担不同义务。

3.2 零基础例子

用户在 CMP 中选择：拒绝个性化广告
App 代码：adSdk.setUserConsent(false)
Wrapper：需要继续调用 Google/Unity/... 的对应隐私 API
广告 SDK：初始化与数据采集逻辑必须尊重该值

可能出现四种典型失败：App 没调用；App 把 false 写成 true 或硬编码；SDK 默认值允许采集；Wrapper 只配置自己，没有配置内部 SDK。PICOSCAN 的价值，就是把这些“工程常识”转成可自动扫描的技术模式。

3.3 四项隐私原则

原则	责任方	白话解释	可检查现象
完整、用户控制的配置	宿主 App	所有 PICO SDK 都应按真实用户偏好配置	遗漏调用、硬编码、与偏好冲突
Privacy by Default	PICO SDK	开发者忘记配置时也应默认保护隐私	默认开启采集或出售
承诺明确且兑现	PICO SDK	调用拒绝 API 后，含义不能模糊，行为必须兑现	拒绝后仍采集、不同 API 语义冲突
完整隐私委托	Wrapper	Wrapper 必须把配置传给全部内部 SDK	漏传、错传、仅部分支持

为什么需要原则 法规语言太高层，代码分析器无法直接搜索“知情同意”。论文先把价值要求变成角色原则，再把原则变成 PVP 代码模式。

3.4 PICOSCAN 系统分工

- **PICO METADB**: 知识库，记录 SDK 包名、隐私 API 签名、支持的法规/目的、参数语义、默认配置以及 Wrapper 关系。它不只是 SDK 名字清单，而是自动分析所需的可执行元数据。

- **PICO Usage Inspector (PUI)** : 在 App 代码中定位 SDK 和隐私 API 调用, 区分调用来自宿主代码还是 Wrapper, 检查宿主 App 责任。
- **PICO SDK Inspector (PSI)** : 进入已编译进 App 的 SDK 代码, 检查隐私 API 内部实现、默认值和 Wrapper 传播关系。
- **PICO Violation Patterns (PVP)** : 把四项原则细化为九类可以在代码或运行时识别的违规模式。

3.5 自动化静态与动态分析如何配合

主流程是静态分析: 先按包名识别 SDK, 再匹配隐私 API 签名、分析调用者、参数值和调用关系。遇到“默认配置实际是什么”“调用拒绝 API 后内部状态是什么”等静态信息不足的问题, 再通过 Hook 或运行时跟踪读取值。

```
for app in app_dataset:
    sdk_set = identify_sdks(app, PICO_METADB)
    calls = find_privacy_api_calls(app, sdk_set)
    check_missing_or_conflicting_calls(calls)          # PUI
    inspect_sdk_defaults_and_api_implementation(app)  # PSI
    inspect_wrapper_propagation(app)                  # PSI
    emit_PVP_evidence(app)
```

大规模测试不是研究者手动点 48,305 个 App, 而是先用静态分析批量筛查。只有某些无法静态确定的 PVP、工具准确率验证和案例确认才使用动态分析或人工核验。

3.6 实验结果应怎样读

- 48,305 个 App 中有 24,844 个集成 PICO SDK; 其中 7,880 个 (31.7%) 至少包含一种 PICO 风险。
- 31.5% 的相关 App 没有调用隐私 API, 而 43.1% 的 PICO SDK 默认配置允许处理个人数据。
- 259 个 App 出现硬编码为允许或与用户偏好冲突的配置。
- 16 个 SDK 在拒绝配置下仍存在含义或行为上的歧义, 覆盖 14,869 个 App。
- 22 个 PICO Wrapper 中有 12 个 (54.5%) 没有完整传递配置。

不要误讲 PICOSCAN 报告的是技术层面的隐私风险，不等同于法院已经认定违法。法规是否适用还取决于公司规模、地域、法律基础等非技术信息。

3.7 局限性与可追问点

- 四项原则和九类 PVP 不是穷尽集合，后端配置、数据删除和跨设备同步仍在范围之外。
- 静态分析难覆盖动态加载、反射和运行时触发；工具也不抗代码混淆，因此测得风险更接近下界。
- 客户端配置可能被服务端配置覆盖，两者冲突需要新的跨端分析。
- 技术风险到法律违规之间仍需要法律专家和主体信息。

3.8 给老师讲解的五分钟卡片

开场（30 秒）：现在的 App 大量依赖广告和统计 SDK。SDK 提供隐私 API，看起来能帮助合规，但配置需要穿过 App、Wrapper 和多个下游 SDK，任何一层都可能出错。

核心问题（30 秒）：PICO SDK 是否真的提升隐私合规？风险来自 App 使用、SDK 实现还是 Wrapper 委托？

方法（2 分钟）：作者先提出四项角色原则，再建立 PICO METADB，把原则细化为 PVP；PUI 检查 App 调用，PSI 检查 SDK 内部与 Wrapper 传播，静态分析为主、动态分析补充。

结果（1 分钟）：超过三成集成 PICO SDK 的 App 至少有一种风险；漏调隐私 API 和 Wrapper 传播失败非常突出。

评价（1 分钟）：贡献在于把法律/隐私原则转成可执行分析，但客户端、混淆和法律适用性仍是边界。

3.9 适合你的简化 Demo

不要一开始反编译数万 App。先做一个可解释的小系统：准备三个示例 Android 模块——宿主 App、Wrapper、FakeAdSDK；用 JSON 维护 SDK 隐私 API 知识库；用 Jadx 输出或源代码 AST 查找初始化与隐私配置调用；输出“漏配、硬编码、未传播”三类报告。

```
knowledge_base.json
FakeAdSDK.init()
FakeAdSDK.setConsent(boolean)
safe_default = false
```

规则 A：出现 `init()`，但控制流之前没有 `setConsent()` → 漏配

规则 B：`setConsent(true)` 为常量且无用户状态来源 → 可疑硬编码

规则 C：`Wrapper.setConsent()` 未调用内部 SDK 对应 API → 委托失败

3.10 阅读后必须能回答

- 为什么 PICO METADB 不只是 SDK 名单？哪些字段会直接决定分析结果？
- 为什么不能只检查 App 有没有调用隐私 API？
- 静态分析已经覆盖全代码，为什么还需要动态分析？
- 31.7%代表违法率吗？为什么不是？
- 如果加入第五条原则，你会选择服务端一致性、数据最小化还是可审计性？如何把它变成代码模式？

正式论文页面：<https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-yifan>

论文 PDF：<https://www.usenix.org/system/files/usenixsecurity24-zhang-yifan.pdf>

第四部分 精读二：GPTDroid——LLM 驱动移动 GUI 动态测试

论文：Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. ICSE 2024。

先记住一句话 GPTDroid 把 GUI 测试改写成 LLM 与 App 持续问答的过程，并用功能级记忆避免只在局部页面乱点。

GPTDroid: LLM与App的闭环对话

LLM负责语义决策，自动化框架负责可靠执行与证据采集



图3 GPTDroid 闭环：状态提取、语义决策、可靠执行与反馈

4.1 摘要原意精译

传统随机或学习型 GUI 测试容易覆盖率低、泛化差并依赖训练数据。GPTDroid 把自动 GUI 测试建模为问答任务：系统把当前页面和控件信息交给 LLM，LLM 生成可执行操作，框架执行后再把新状态反馈给 LLM。作者还设计功能感知记忆，让模型记住已探索页面、控件、功能完成状态和近期操作。论文在 93 个 Google Play App 上评估，活动覆盖率相对最佳基线提高 32%，发现缺陷数量提高 31%；另发现 53 个新缺陷，其中 35 个得到确认和修复。

4.2 Appium/UIAutomator 和 LLM 的分工

组件	负责什么	不负责什么
UIAutomator/Appium	读取控件树、点击、输入、滚动、截图	不真正理解“隐私设置”或“下一步最有价值”
ADB/设备控制	安装、启动、重置、日志与崩溃信息	不决定测试策略
LLM	理解控件语义、选择动作、生成有效输入	不直接保证点击成功，也不应独自判定所有漏洞
测试框架	维护状态、约束输出、重试、覆盖率和证据	不能靠一个 Prompt 自动获得可靠性

4.3 双循环结构

外循环完成一次标准交互：读取当前 GUI 上下文，生成 Prompt，解析 LLM 答案，执行动作并获得反馈。内循环维护长期测试知识：记录访问过的 Activity 和 Widget，并询问 LLM 当前正在测试哪个功能、是否完成，再将功能进度编码进下一轮 Prompt。

```
while budget_not_exhausted:
    ui = extract_gui_context(device)
    memory = summarize_test_history()
    prompt = build_prompt(ui, memory, output_schema)
    action = parse_and_validate(llm(prompt))
    result = execute(action)
    update_memory(ui, action, result)
    if action_failed: send_feedback_and_reselect()
```

4.4 GUI 上下文到底包含什么

- App 级：包名、可见 Activity、整体功能背景。
- 页面级：当前 Activity 或页面名称、页面层级结构。
- 控件级：text、resource-id、class、clickable、父节点和兄弟节点文本。
- 历史级：已测试功能、已覆盖 Activity、最近操作和失败反馈。

作者不是把原始 XML 完全丢给模型，而是提取、排序和语言化关键字段。这样做同时减少 Token、提升可理解性，也使输出与具体控件更容易绑定。

4.5 如何把 LLM 文字变成可执行动作

论文通过少样本示例和固定输出模板约束模型。例如动作被限制为 click、double-click、long press、scroll 和 input，并要求返回控件名或结构化字段。解析器再把控件名映射到真实节点。执行失败时，框架反馈“当前页面没有该控件”，要求模型重新选择。

```
Function: Add income  
Status: No  
Operation: Click  
Widget: ADD INCOME
```

或

```
Widget: Price | Input: 3500  
Widget: Title | Input: salary  
Operation: Click | Widget: Submit
```

4.6 功能感知记忆为什么是核心

随机工具容易在设置页、列表页或返回路径中循环。GPTDroid 不仅记录“点过哪些按钮”，还追踪“新增收入这个功能是否完成”。论文消融实验中，完整系统 Activity 覆盖率为 0.75；移除 GUI 上下文降到 0.17，移除功能记忆降到 0.34。这说明性能来自结构化上下文和记忆机制，而不仅是调用了一个强 LLM。

4.7 如何评价实验

- 覆盖率提高说明探索更广，但覆盖率本身不等于发现安全/隐私问题。
- 53 个新缺陷、35 个被确认和修复，比单纯覆盖率更能说明实用性。
- 使用 LLM 会引入模型版本、温度、网络、成本和不可重复性问题；复现时应固定 Prompt、模型和随机参数，并保存全部交互日志。
- 登录、验证码、付费、权限弹窗和第三方页面仍可能阻塞全自动探索。

4.8 与 DIULENS 的直接连接

GPTDroid 的目标是一般功能覆盖；DIULENS 把目标改成“尽快找到 CMP 入口并完整探索隐私选项”。你可以把 GPTDroid 理解为通用导航发动机，而 DIULENS 增加了 CMP 知识、隐私优先级、证据记录 and DIU 规则 Oracle。

GPTDroid	DIULENS 中的对应升级
一般 GUI 上下文	CMP 关键词、隐私设置语义、截图和 SDK 证据
功能感知记忆	CMP 入口路径、已访问层级、已尝试同意分支
覆盖率目标	CMP 发现率、界面层级完整性、规则证据完整性
崩溃/覆盖 Oracle	DIU 四规则 Oracle

4.9 给老师讲解的五分钟卡片

问题：传统 Monkey 会随机点，学习型方法泛化有限，难以理解页面功能。

洞察：GUI 天然包含文本和语义，可以把测试建模为 LLM 与 App 的持续问答。

方法：外循环完成状态—决策—执行；内循环维护 Activity、Widget 和功能完成状态；固定模板将答案转成可执行命令。

结果：覆盖和缺陷发现均优于基线，消融实验说明 GUI 上下文和功能记忆不可缺少。

批判：LLM 负责策略并不等于系统完全可靠，工具执行、输出验证、失败恢复和测试 Oracle 仍是核心工程。

4.10 适合你的简化 Demo

在现有 Android Demo 上增加 AgentExplorer：用 Appium 读取 XML 和截图；过滤不可交互节点；把控件列表、目标“找到 Privacy/Consent 设置”和历史动作发给 LLM；要求 JSON 输出；执行并记录每一步。先不做漏洞判断，只演示“AI 自动找到 CMP 并进入第二层选项”。

```
{  
  "goal": "find_consent_settings",  
  "action": "click",  
  "target": {"resource_id": "privacy_settings", "text": "Privacy"},  
  "reason": "该入口最可能进入 CMP 管理页"  
}
```

ICSE 正式页面: <https://conf.researchr.org/details/icse-2024/icse-2024-research-track/180/Make-LLM-a-Testing-Expert-Bringing-Human-like-Interaction-to-Mobile-GUI-Testing-via->

作者公开预印本: <https://arxiv.org/pdf/2310.15780>

第五部分 精读三：IRIS——LLM 辅助静态污点分析

论文：IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. ICLR 2025。

先记住一句话 IRIS 不让 LLM 独自读完整仓库猜漏洞，而让 LLM 补充 Source/Sink 语义，再由 CodeQL 做跨函数数据流，最后让 LLM 辅助过滤误报。

IRIS：神经符号静态分析流水线

让 LLM 补语义，让程序分析负责跨函数数据流



图4 IRIS 四阶段：候选、规范、数据流和上下文过滤

5.1 摘要原意精译

传统静态漏洞分析高度依赖人工编写的规范，容易因为缺少第三方 API 的 Source、Sink 或 Sanitizer 定义而漏报；同时，路径近似又会产生大量误报。IRIS 提出神经符号方法，将 LLM 与静态分析系统结合，用 LLM 推断污点规范和进行上下文分析，用 CodeQL 负责全仓库数据流。作者构建含 120 个真实 Java 漏洞的 CWE-Bench-Java；CodeQL 检测到 27 个，IRIS 配合 GPT-4 检测到 55 个，并发现 4 个此前未知漏洞。

5.2 三个基础概念

Source（来源）：`request.getParameter("path")`

↓ 用户可控字符串在函数之间传播

Sanitizer（净化）：`canonicalize(path)` + 检查必须位于安全目录

↓ 如果没有有效净化

Sink（危险点）：`new FileInputStream(path)`

静态污点分析要回答：是否存在一条从 Source 到 Sink、且中间没有经过有效 Sanitizer 的路径。不同 CWE 需要不同规范。例如路径穿越、命令注入、跨站脚本和代码注入的 Sink 完全不同。

5.3 为什么 CodeQL 自己还不够

- 第三方库持续变化，人工无法为每个 API 维护完整的 Source/Sink 规范，导致漏报。
- 静态分析为了不漏掉可能路径，会进行过近似，产生许多实际不可利用的告警。
- 单独 LLM 又不擅长在几十万行代码中稳定追踪跨函数数据流。

神经符号 “神经”指 LLM 的语义归纳，“符号”指 CodeQL 数据流图和查询规则。两者不是竞争关系，而是分工。

5.4 四阶段流水线

阶段	传统工具	LLM 任务	输出
1 候选提取	构建 CodeQL 数据库，枚举外部 API 和内部参数	无或很少	候选 API 集合
2 规范推断	提供签名、文档和 CWE 信息	标注 Source/Sink 及位置	可执行污点规范
3 污点分析	CodeQL 搜索跨过程未净化路径	不负责路径搜索	候选漏洞路径
4 上下文过滤	提供代码路径与上下文	判断可利用性并解释	过滤后的告警与说明

5.5 一个具体的代码注入例子

论文展示 cron-utils 中的代码注入：用户输入进入解析器；异常消息包含该输入；消息随后传给会解释 Java EL 表达式的 API。如果攻击者构造恶意表达式，就可能执行危险代码。常规 CodeQL 没有把那个第三方 API 识别成 Sink，因此数据流存在却没有产生告警。IRIS 通过 LLM 补上 Sink 语义，CodeQL 才得以发现完整路径。

5.6 实验结果该怎样正确解读

指标	CodeQL	IRIS + GPT-4	解释
检测到的 120 个真实漏洞	27	55	召回明显提高，但仍漏掉 65 个
平均误发现率 FDR	约 90%	84.82%	有所改善，但绝对误报仍然很高
新发现漏洞	—	4	说明可在真实项目产生价值

关键批判 “比 CodeQL 好”不等于“已经能自动取代安全工程师”。84.82%的 FDR 意味着多数告警仍需进一步验证。

5.7 局限性

- 仍有大量漏洞无法检测，静态分析难表达运行时副作用、复杂 gadget chain 等行为。
- 规范推断和误报过滤需要大量 LLM 调用，分析成本可能较高。
- 论文主要验证 Java，是否能直接推广到 C/C++、Python 或 Android 字节码并不确定。
- LLM 可能漏标 taint propagator，导致数据流边缺失；也可能在上下文过滤中误判。
- 输出报告与开发者真正需要的修复信息之间仍有距离。

5.8 与移动隐私合规如何连接

把 IRIS 迁移到移动隐私方向，可以让 LLM 从 SDK 文档和代码推断：哪些 API 返回设备标识符、定位和联系人（Source），哪些 API 上传网络或写共享存储（Sink），哪些配置 API 可能阻断传播（Sanitizer/Guard）。静态分析负责寻找跨 App 代码和 SDK 代码的数据流，再用动态网络或 Hook 证据复核。

LLM 从 SDK 文档推断隐私规格

```
AdvertisingIdClient.getAdvertisingIdInfo → Source(identifier)
Analytics.track / HttpClient.execute      → Sink(network)
setUserConsent(false)                    → Guard/Configuration
```

CodeQL/FlowDroid 验证: Source 是否能在拒绝同意后到达 Sink

5.9 给老师讲解的五分钟卡片

问题: 静态分析依赖人工规范且误报高; LLM 单独分析又无法稳定跨越整个仓库。

方法: 先由 CodeQL 提取候选 API, LLM 根据 CWE 标注 Source/Sink, CodeQL 运行污点查询, 再由 LLM 结合路径上下文过滤误报。

结果: 在 120 个 Java 真实漏洞上从 27 提高到 55, 并发现 4 个新漏洞。

评价: 这是典型的神经符号组合; 优势来自分工, 但误报、成本、语言泛化仍然突出。

5.10 适合你的简化 Demo

先不用完整 CodeQL。准备一个小型 Java 项目, 手工给出候选 API 列表, 让 LLM 按 JSON 标注 Source/Sink; 使用简单的调用图或 Semgrep 规则找直接路径; 最后把代码片段交给 LLM 解释。Demo 重点展示“规范推断—确定性分析—语义复核”三段式架构。

ICLR 正式页面:

https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html

论文 PDF:

https://proceedings.iclr.cc/paper_files/paper/2025/file/582d4e27fa24168f3af1f4582655034b-Paper-Conference.pdf

第六部分 扩展论文导读：知道为什么读、重点读哪里

6.1 PESP : Towards Privacy-Preserving Social-Media SDKs on Android

出处：USENIX Security 2024

研究同一 App 进程中的恶意/不可信库如何获取社交 SDK 掌握的丰富用户数据，论文称为 Cross-Library Data Harvesting (XLDH)，并提出隐私保护设计 PESP。

值得阅读的部分：

- 精读威胁模型：攻击者是谁，为什么 Android 沙箱不能隔离同一 App 内的库。
- 精读 PESP 如何限制库间数据访问，以及兼容性和性能开销。
- 和 PICOSCAN 比较：PICOSCAN 检查配置是否正确，PESP 解决即使配置正确仍可能发生的库间数据窃取。

阅读时思考：

- SDK 运行在同一进程时，谁应该拥有数据访问权限？
- PESP 的隔离能否覆盖反射、native code 和动态加载？

正式页面：<https://www.usenix.org/conference/usenixsecurity24/presentation/lu-haoran>

6.2 隐蔽标识符：Exploring Covert Third-party Identifiers through External Storage in the Android New Era

出处：USENIX Security 2024

研究追踪 SDK 把自生成标识符藏在外部存储中，从而跨 App 识别用户。作者用内核级插桩与 UI 自动测试记录 8,000 个 App 的文件操作，识别出 17 个相关 SDK。

值得阅读的部分：

- 精读动态分析管线：如何自动启动 App、触发 SDK、记录文件操作并做 SDK 归因。
- 精读标识符判定标准和 Scoped Storage 绕过案例。
- 把它作为“运行时证据”的范例，补足 PICOSCAN 的静态分析视角。

阅读时思考：

- 如何证明文件中的字符串真的是跨 App 标识符，而不是缓存？
- 自动 UI 探索不足会造成怎样的漏报？

正式页面： <https://www.usenix.org/conference/usenixsecurity24/presentation/dong-zikan>

6.3 InconPreter : What's Done Is Not What's Claimed

出处：NDSS 2025

从用户视角提取 App 行为并解释“用户预期”与“程序真实行为”的不一致。工具在 10,878 个 App 中，从 413 个 App 识别 1,664 项风险行为。

值得阅读的部分：

- 精读用户相关行为如何表示，而不只是 API 调用。
- 精读风险解释如何生成和验证。
- 与 DIULENS 对比：两者都研究界面/用户理解与真实行为的差距，但 DIULENS 聚焦 CMP 有效同意。

阅读时思考：

- 用户预期由谁定义？不同用户是否有不同合理预期？
- 自然语言解释准确是否等于底层分析准确？

正式页面： <https://www.ndss-symposium.org/ndss-program/symposium-2025/>

6.4 LLift : Enhancing Static Analysis for Practical Bug Detection

出处：OOPSLA 2024

让静态分析先产生 Use-Before-Initialization 候选，再用面向缺陷的 Prompt、后置约束和程序上下文减少误报。正式论文报告确认 4 个此前未知的 Linux 内核缺陷。

值得阅读的部分：

- 精读为什么通用 Prompt 不足，需要 bug-specific modeling。

- 精读 post-constraint guidance 和多轮验证。
- 与 IRIS 比较：IRIS 让 LLM 补规范并过滤路径，LLift 更聚焦一种具体缺陷的告警判定。

阅读时思考：

- LLM 判断“不会发生”的路径是否足以安全地过滤告警？
- 怎样保存证据以便人工审计？

正式页面：<https://2024.splashcon.org/details/splash-2024-oopsla/18/Enhancing-Static-Analysis-for-Practical-Bug-Detection-An-LLM-Integrated-Approach>

6.5 CoverUp : Effective High Coverage Test Generation for Python

出处：FSE 2025

LLM 生成测试后立即执行覆盖率分析，再把未覆盖行、分支和失败反馈给 LLM，循环改进。模块级行与分支覆盖率中位数达到 80%，论文强调反馈循环贡献显著。

值得阅读的部分：

- 精读 coverage-guided loop，而不是只看 Prompt。
- 精读失败测试、flaky 测试和测试隔离如何处理。
- 它不是移动安全论文，但最适合练习 Agent—Tool—Feedback 架构。

阅读时思考：

- 高覆盖率是否一定能发现真实缺陷？
- 如果被测代码本身错误，LLM 会不会生成巩固错误行为的测试？

正式页面：<https://conf.researchr.org/details/fse-2025/fse-2025-research-papers/112/CoverUp-Effective-High-Coverage-Test-Generation-for-Python>

6.6 MAdroid : Breaking Single-Tester Limits

出处：ICSE 2026

针对聊天、通话、直播等多用户功能，使用 Coordinator、Operator 和 Observer 三类智能体协同控制多个设备。论文在 41 项任务中完成 82.9%，并在回归测试中发现 11 个多用户交互缺陷。

值得阅读的部分：

- 精读三类 Agent 的职责和消息协议。
- 精读如何同步多个设备状态以及 Observer 如何判定偏差。
- 把它看成 GPTDroid 单设备闭环向多设备协作的升级。

阅读时思考：

- 多个 Agent 的错误会相互放大还是相互纠正？
- 多用户任务的 Oracle 如何定义？

正式页面：<https://conf.researchr.org/details/icse-2026/icse-2026-research-track/18/Breaking-Single-Tester-Limits-Multi-Agent-LLMs-for-Multi-User-Feature-Testing>

第七部分 横向比较：把论文放进同一张地图

7.1 技术和研究对象对比

论文	对象	主要方法	Oracle/目标	复现成本
DIULENS	CMP 与 SDK	静态+动态+LLM GUI	DIU 规则	高
PICOSCAN	App/SDK/Wrapper	静态为主+动态补充	PVP	中
GPTDroid	移动 GUI	LLM Agent+UI 自动化	覆盖/崩溃	中
IRIS	Java 仓库	LLM+CodeQL 污点分析	CWE 路径	较高
PESP	SDK 间数据	系统隔离与实现	XLDH 阻断	高
隐蔽标识符	追踪 SDK	内核监控+UI 测试	跨 App 标识符	高
InconPreter	用户相关行为	行为抽取+解释	预期不一致	较高
LLift	Linux 内核告警	静态分析+LLM 过滤	UBI 缺陷	较高
CoverUp	Python 测试	覆盖反馈+LLM	覆盖率	低
MAdroid	多用户 App 功能	多 LLM Agent+多设备	任务/行为一致	很高

7.2 三种“知识库”不要混淆

系统	知识是什么	知识怎样被使用
DIULENS	CMP 列表、SDK 注册信息、CMP 指南、DIU 规则、UI 探索历史	识别真实组件、指导 GUI 探索并判断规则
PICOSCAN	SDK 包名、隐私 API、参数语义、法规支持、默认值、Wrapper 关系	把 APK 代码匹配到 PVP 并做大规模扫描
IRIS	CWE 定义、API 签名/文档、代码上下文；部分知识隐含在 LLM 中	推断 Source/Sink 规范并过滤告警

高效知识库建设的共同原则是：先确定分析器需要回答的问题，再只收集能够改变规则判断的字段；为每条知识记录来源、版本、适用平台和置信度；用自动抓取扩大规模，用人工核验保证种子质量。

7.3 自动化并不等于完全无人值守

自动批处理：下载/安装 → 识别组件 → 静态扫描 → 动态探索 → 规则判定 → 报告

人工介入点：知识库种子、工具准确率标注、疑难登录/验证码、案例确认、法律判断

大规模研究通常由电脑自动跑绝大多数 App，最后自动汇总统计；但在管线设计、标注集、误报确认和责任归因环节仍需要研究者。论文写作中必须把“自动分析结果”和“人工确认结果”分开报告。

7.4 研究质量检查清单

- 问题定义：研究的是技术风险、政策违规还是法律违法？三者不能混用。
- 数据集：App 从哪里来、何时下载、是否去重、是否代表真实生态？
- Ground Truth：谁标注、标注多少、是否双人复核、分歧如何处理？
- 覆盖：动态探索走了哪些路径，未登录或未触发功能如何影响漏报？
- LLM：模型/版本/Prompt/温度是否公开，输出是否有 Schema 和复核？
- 准确率：只报 Precision 不够，还应讨论 Recall、误报和漏报来源。
- 可复现：代码、知识库、数据、脚本、日志与成本是否公开？

7.5 跨论文核心思考题

- 如果用户拒绝后 SDK 仍访问设备标识符，问题可能发生在 CMP、App、Wrapper、SDK 还是后端？需要哪些证据才能归因？
- 规则应该来自法律条文、平台政策、SDK 文档还是研究者常识？不同来源冲突时怎么办？
- 能否把 Consent Signal 视作污点分析中的一种控制变量，而不是数据 Source？

- LLM 作出的 UI 判断或 Source/Sink 标注如何被确定性工具验证？
- 静态分析的“全代码覆盖”和动态分析的“真实执行证据”，哪个更适合证明合规？
- 一个系统提高覆盖率但没有更好的 Oracle，是否真的更善于发现隐私问题？
- 大规模自动化测试中，如何识别登录、网络失败和地区差异造成的假阴性？
- 同一个 SDK 不同版本、不同远程配置和不同地区行为不一致，知识库如何版本化？

第八部分 八周学习与实践路线

周	学习目标	阅读/实践产出
第 1 周	补齐 Android 组件、APK、SDK、ADB/Appium	能手动安装、重置、导出 UI XML 并解释控件字段
第 2 周	理解隐私同意与 DIULENS	画出 CMP—Signal—SDK 链路；复述四条 DIU 规则
第 3 周	精读 PICOSCAN	整理四原则、九类 PVP 思想；做 JSON 版 SDK 知识库
第 4 周	做 PICOSCAN 简化 Demo	检测漏调、硬编码、Wrapper 漏传三类问题
第 5 周	精读 GPTDroid	实现 UI 状态→LLM JSON 动作→Appium 执行的闭环
第 6 周	理解污点分析并精读 IRIS	用小 Java 例子画 Source—Sink 路径；运行 CodeQL 或 Semgrep
第 7 周	组合静态与动态证据	对同一示例 App 输出 SDK 清单、UI 路径、事件和规则报告
第 8 周	准备研讨汇报	10—15 页 PPT、5 分钟 Demo、问题清单和局限性页

8.1 推荐的最终 Demo 架构

知识库层: SDK 包名 / 隐私 API / 参数语义 / CMP 关键词 / 规则

↓

静态层: APK 或示例源码 → SDK 识别 → 隐私 API 与初始化顺序

↓

动态层: Appium → CMP 路径 → 截图/XML → 点击前后事件

↓

LLM 层: UI 目标选择 + 页面语义归类 + 证据解释

↓

规则层: DIU + 简化 PVP → 结构化 JSON 报告 → 可视化展示

实现顺序应是：先确定性规则、再自动化 UI、最后接 LLM。这样即使 LLM 不可用，Demo 仍能展示静态扫描和固定路径测试；接入 LLM 后只是提升泛化和探索能力。

8.2 可形成的小论文问题

- CMP 同意选择能否自动映射到不同 SDK 隐私 API，并检测配置传播不一致？
- LLM 推断的 SDK 隐私规范能否通过动态 Hook 自动验证并回写知识库？
- 以覆盖率、CMP 层级覆盖和规则证据完整度联合奖励，能否改进 LLM GUI 探索？
- 同一 App 在地区、语言和版本变化下，CMP 界面与 SDK 行为是否一致？
- 能否构建可审计 Agent：每个判断都必须引用截图、UI 节点、调用栈或网络事件？

附录 A 给老师讲论文的通用模板

A.1 十分钟讲解结构

时间	内容	必须说清楚
0:00–1:00	背景与具体例子	现实问题是谁遇到的，为什么现有方法不够
1:00–2:00	研究问题与威胁模型	输入、攻击者/责任方、输出
2:00–5:30	系统设计	模块分工、数据流、LLM 和传统工具边界
5:30–7:30	实验	数据集、基线、指标、最关键两项结果
7:30–9:00	案例/Demo	展示一条完整证据链
9:00–10:00	局限与想法	一个真实限制和一个可做的改进

A.2 一页阅读笔记模板

- 论文信息：完整题目、作者、会议/期刊、年份、正式链接。
- 一句话问题：现有系统在哪个具体场景失败？
- 威胁/责任模型：谁控制什么？谁应该承担什么义务？
- 输入与输出：系统吃什么数据，产生什么证据或告警？
- 核心洞察：为什么旧方法做不到，作者的新观察是什么？
- 模块链路：每个模块的输入、处理、输出和依赖。
- 实验：数据集、基线、指标、主结果、消融实验。
- 局限：论文自述限制 + 你自己发现的限制。
- 复现：最小可运行子集、依赖、数据和验收标准。
- 我的问题：至少三个能与老师讨论的问题。

附录 B 常见老师追问与回答框架

追问	回答框架
为什么要用 LLM?	指出具体语义缺口，例如控件含义或第三方 API 规范；再说明传统工具负责验证，避免回答“因为 LLM 很强”。
为什么不用纯动态/纯静态?	说明覆盖面与真实性的取舍，以及两者如何互相补证。
规则是谁设计的?	追溯到法规/标准/文档和角色责任，再说明规则不完备及适用边界。
自动化程度到底多高?	分开说明批处理、人工种子、人工验证和法律判断。
结果能否证明违法?	通常只能证明技术风险或政策不一致，法律结论需要主体、地域和法律基础。
LLM 幻觉怎么办?	结构化输出、候选集合约束、工具复核、多次采样、一致性检查和证据引用。
Demo 和原系统差多少?	诚实区分概念验证、单 App 实验、跨版本评估和论文级大规模系统。

附录 C 正式来源

DIULENS / IEEE S&P 2026 预印本: <https://xw48.github.io/files/ye2026understanding.pdf>

PICOSCAN / USENIX Security 2024: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-yifan>

PESP / USENIX Security 2024: <https://www.usenix.org/conference/usenixsecurity24/presentation/lu-haoran>

Covert Third-party Identifiers / USENIX Security 2024: <https://www.usenix.org/conference/usenixsecurity24/presentation/dong-zikan>

GPTDroid / ICSE 2024: <https://conf.researchr.org/details/icse-2024/icse-2024-research-track/180/Make-LLM-a-Testing-Expert-Bringing-Human-like-Interaction-to-Mobile-GUI-Testing-via->

IRIS / ICLR 2025: https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html

InconPreter / NDSS 2025: <https://www.ndss-symposium.org/ndss-program/symposium-2025/>

LLift / OOPSLA 2024: <https://2024.splashcon.org/details/splash-2024-oopsla/18/Enhancing-Static-Analysis-for-Practical-Bug-Detection-An-LLM-Integrated-Approach>

CoverUp / FSE 2025: <https://conf.researchr.org/details/fse-2025/fse-2025-research-papers/112/CoverUp-Effective-High-Coverage-Test-Generation-for-Python>

MAdroid / ICSE 2026: <https://conf.researchr.org/details/icse-2026/icse-2026-research-track/18/Breaking-Single-Tester-Limits-Multi-Agent-LLMs-for-Multi-User-Feature-Testing>

最后提醒 讲论文时最有价值的不是背数字，而是能沿着“问题—证据—模块—规则—实验—局限”完整解释一次，并知道每个结论由什么证据支撑。